

Query & Buy - Technical Audit

AI-powered marketplace for the UAE. Prepared for technical due diligence, infrastructure planning, and investor review.

Report date: 30 June 2026. Scope: grounded only in the actual repository (Next.js 15 + Supabase + Gemini). Every claim below is verified against source files; cost projections are clearly-labelled estimates built on stated assumptions, not measured production data.

Legend: [IMPLEMENTED] = present and working in code. [PARTIAL] = scaffolding/table exists but flow incomplete. [PLANNED] = templated/stubbed only.

Part 1 - Technology Stack

Frontend

- **Framework:** Next.js 15 (App Router, React Server Components, Server Actions). [IMPLEMENTED]
- **Language:** TypeScript 5, strict mode, path aliases (@/*). [IMPLEMENTED]
- **UI library:** shadcn/ui built on Radix UI primitives (button, card, dialog, select, dropdown, avatar, input, etc.). [IMPLEMENTED]
- **Styling:** Tailwind CSS v4 (no config file; inline @theme tokens in globals.css; emerald + champagne-gold design system; OKLCH colors; light/dark). [IMPLEMENTED]
- **Icons:** lucide-react (tree-shaken via experimental optimizePackageImports). [IMPLEMENTED]
- **Forms:** Native HTML forms + Server Actions. No react-hook-form. [IMPLEMENTED, lightweight]
- **State management:** React local state (useState) + server state via RSC. No Redux/Zustand/global store. [IMPLEMENTED, minimal]
- **Tables:** None (no TanStack Table); data rendered as cards/lists. [NOT USED]
- **Charts:** None (no chart library installed). [NOT USED]
- **Validation:** Inline length/enum checks in server actions + Postgres constraints + RLS. No Zod. [IMPLEMENTED]
- **Routing:** Next.js App Router (file-based, route groups, dynamic segments). [IMPLEMENTED]
- **Other:** next-themes (dark mode), sonner (toasts), class-variance-authority + tailwind-merge + clsx (cn util), tw-animate-css. Fonts: Geist (sans) + Fraunces (serif) via next/font. [IMPLEMENTED]

Backend

- **Database:** Supabase PostgreSQL (27 tables; PostGIS geography; pgvector 1024-dim embeddings; tsvector full-text search; soft-deletes; denormalized counters). [IMPLEMENTED]
- **Authentication:** Supabase Auth (GoTrue) via @supabase/ssr - email/password + Google OAuth (PKCE) + email OTP confirm. [IMPLEMENTED]
- **Storage:** Supabase Storage - two public buckets: avatars (2 MB) and listing-images (5 MB), owner-scoped by path prefix. [IMPLEMENTED]
- **APIs:** PostgREST (auto REST over Postgres) used by the Supabase client; 2 custom Route Handlers (/auth/callback, /auth/confirm). Primary write path is Server Actions. [IMPLEMENTED]
- **ORM:** None. Raw @supabase/supabase-js queries (typed by hand). [BY DESIGN]
- **Server actions:** 50+ across auth, account, sell, search, messages, orders/offers, reviews, reports, saved-searches, notifications, admin, feedback. [IMPLEMENTED]
- **Middleware:** Session refresh (JWT rotation) on every request + route protection for /account, /sell, /messages, /favorites, /saved-searches, /notifications, /admin. [IMPLEMENTED]

AI

- **Provider:** Google Gemini via REST (generativelanguage.googleapis.com/v1beta). Key from GEMINI_API_KEY (server-only). [IMPLEMENTED]

- **Abstraction layer:** AiProvider interface (lib/ai/provider.ts) selected by AI_PROVIDER env var; getProvider() factory. OpenAI and Claude branches are templated but commented out. [IMPLEMENTED abstraction; alternate providers PLANNED]

- **Model used:** gemini-2.5-flash (single model for all three features). JSON-constrained output via responseSchema. [IMPLEMENTED]

- **Image processing:** Client-side downscale to 1024px JPEG q0.7 -> base64 (lib/ai/image-client.ts); images sent inline to the model for vision analysis. [IMPLEMENTED]

- **AI features implemented:** (1) Listing draft from photos - product ID, condition, category, attributes, 3-tier AED pricing, per-field confidence; (2) Conversational search - natural language to filters, with deterministic heuristic fallback on 429/error; (3) Negotiation advisor - suggested offer/counter, with deterministic fallback. [IMPLEMENTED]

- **AI features planned/not implemented:** OpenAI/Claude providers (templated); AI-based image/content moderation (current moderation is keyword-based + confidence flagging only); embeddings-powered similarity UI (listing_embeddings table + IVFFlat index exist; no live AI write/UI). [PLANNED / PARTIAL]

Infrastructure

- **Hosting:** Vercel (Next.js) - inferred from project; site URL query-and-buy.vercel.app. [IMPLEMENTED via platform]
- **CDN:** Vercel edge for app + image optimization (AVIF/WebP); Supabase Storage public CDN for user images. [IMPLEMENTED]
- **Deployment:** Git push to main (GitHub: mariarojo100/query-and-buy); Vercel auto-deploy. [IMPLEMENTED via platform]
- **Environment variables:** NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, ADMIN_EMAILS, GEMINI_API_KEY, AI_PROVIDER, ANTHROPIC_API_KEY (future), RESEND_API_KEY, EMAIL_FROM, NEXT_PUBLIC_SITE_URL. [IMPLEMENTED]
- **CI/CD:** No GitHub Actions or vercel.json in repo. Quality gate is local (npm run lint = tsc --noEmit, npm run build). Supabase CLI scripts for migrations (db:push/diff/lint). [MISSING - relies on Vercel build]
- **Git:** GitHub; trunk-based on main. [IMPLEMENTED]
- **Monitoring:** No external APM/error-tracking (no Sentry/Datadog). [MISSING]
- **Logging:** Structured JSON logger (lib/logger.ts) with automatic redaction of secrets/PII; sink is console (swappable). [IMPLEMENTED]

Security

- **Authentication:** Supabase Auth, HttpOnly cookie sessions via @supabase/ssr, PKCE OAuth. [IMPLEMENTED]
- **Authorization:** Role matrix (user_roles: user/moderator/admin/super_admin); is_admin()/is_staff()/has_role() SECURITY DEFINER helpers. [IMPLEMENTED]
- **RLS:** Row Level Security enabled on all tables; participant-locked messaging/orders/offers; public-read only for active listings/profiles/reviews; admin tables gated by is_admin(). [IMPLEMENTED]
- **Security headers:** HSTS (2y, preload), X-Frame-Options SAMEORIGIN, X-Content-Type-Options nosniff, Referrer-Policy strict-origin-when-cross-origin, Permissions-Policy (camera/mic/geo off). [IMPLEMENTED]
- **Cookie handling:** Supabase SSR cookie store; session cookies written onto responses in middleware + callback. [IMPLEMENTED]
- **CSRF:** Relies on Next.js Server Actions origin checks; no explicit token layer. [PARTIAL]
- **XSS:** React auto-escaping; JSON-LD via controlled serializer; no dangerouslySetInnerHTML on user content. [IMPLEMENTED]
- **Rate limiting:** No global limiter (no Upstash/middleware throttle). AI has graceful 429 fallbacks. [MISSING - recommended]
- **Input validation:** Server-side length/enum checks + DB constraints + keyword safety screen (weapons/drugs/counterfeit/adult) before publish. [IMPLEMENTED]

SEO

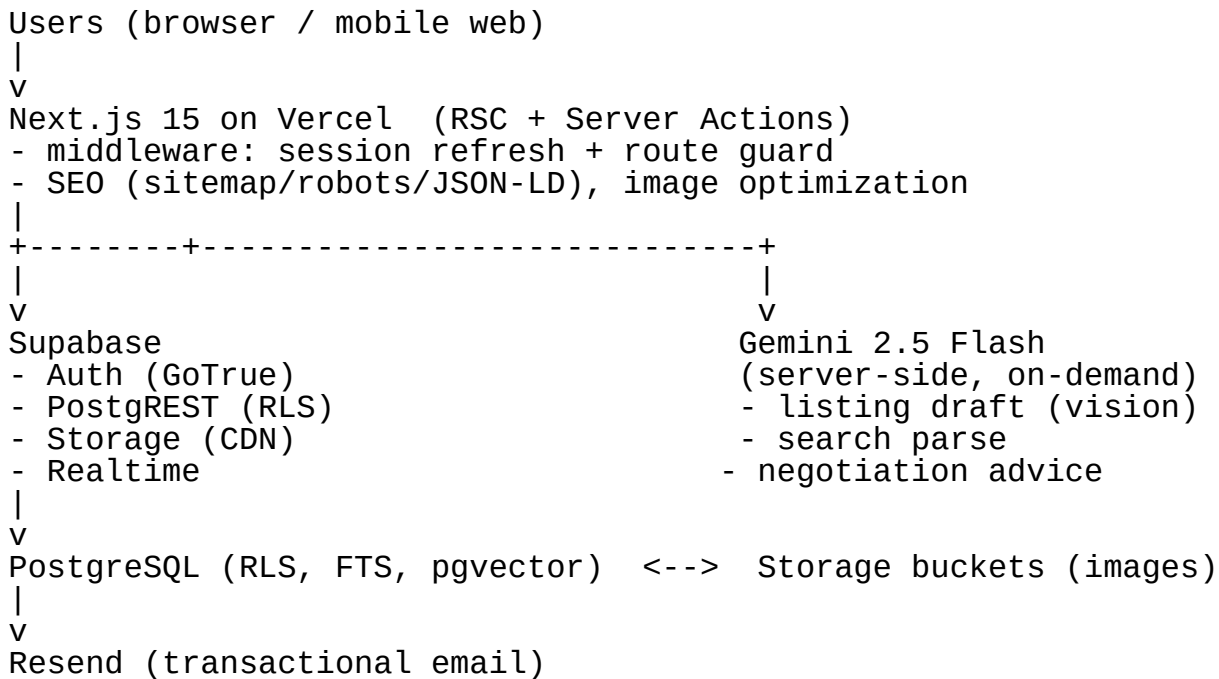
- **Sitemap:** app/sitemap.ts - dynamic (categories, up to 5000 listings, seller profiles), hourly revalidate. [IMPLEMENTED]
- **Robots:** app/robots.ts - allow public, disallow auth-only areas, link sitemap. [IMPLEMENTED]
- **Open Graph:** Configured in root metadata (type, siteName, locale en_AE, url). [IMPLEMENTED]
- **Twitter Cards:** summary_large_image. [IMPLEMENTED]
- **JSON-LD:** Organization, WebSite (with SearchAction), Product (listing pages), BreadcrumbList. [IMPLEMENTED]
- **Canonicals:** metadataBase + alternates.canonical; per-page metadata. [IMPLEMENTED]

Production Features

- **Error pages:** error.tsx, global-error.tsx, not-found.tsx. [IMPLEMENTED]
- **Loading states:** Route-level loading.tsx (listing, profile) + skeletons. [PARTIAL - not on every route]
- **Analytics:** Provider-agnostic wrapper (lib/analytics.ts) with typed events; no-op in prod until a

- provider is wired. [PARTIAL / extensible]
 - **Logging:** Structured JSON + redaction. [IMPLEMENTED]
 - **Email:** Resend (REST API) with 3x exponential-backoff retry + email_failures log + per-category preferences + critical-email bypass. [IMPLEMENTED]
 - **Notifications:** In-app notification center (table + realtime + mark-read) and email. No web push/SMS. [IMPLEMENTED in-app/email; push MISSING]
 - **Admin panel:** /admin - listings, users, moderation/reports, reviews, orders, categories CRUD, marketplace settings, AI moderation log, append-only audit log. [IMPLEMENTED]
-

Part 2 - Architecture



How components communicate:

- **Browser to Next.js:** RSC render over HTTPS; mutations via Server Actions (no hand-written REST). Middleware runs first on every request to refresh the Supabase JWT and guard protected routes.
 - **Next.js to Supabase:** Server components/actions use a server Supabase client (service-role only in admin paths); the browser uses the anon client for reads/realtime. All access is constrained by RLS at the database, so authorization lives in Postgres, not app code.
 - **Next.js to Gemini:** Only three server actions call the model (sell draft, search parse, negotiation). Calls are synchronous, on-demand, JSON-schema-constrained, and never block core flows (heuristic fallbacks on failure).
 - **Images:** Uploaded directly from the browser to Supabase Storage (owner-scoped path), served back via the public Storage CDN and optimized by Vercel's image pipeline.
 - **Realtime:** messages, conversations, offers, orders, reviews, and notifications are published on Supabase Realtime; the client subscribes for live updates.
 - **Email:** Server actions dispatch in-app notifications and, best-effort, Resend emails (preference-gated), logging failures for retry.
-

Part 3 - AI Cost Analysis

Which actions call the AI API (only these three):

- **generateListingDraft** - server action on /sell when the user taps "Generate with AI" after uploading 1-5 photos. Vision + text.
- **parseConversationalSearch** - server action when a natural-language search is submitted (heuristic fallback avoids the call on 429/error).

- **suggestOffer** - server action when a buyer/seller requests negotiation advice (deterministic fallback on error).

Pages/actions that never use AI: homepage feed, browsing, listing detail, favorites, messaging, orders/offers state machine, reviews, notifications, account, admin, auth - all run with zero AI tokens. There is **no caching** of AI results today (each call is fresh).

Per-request token + cost estimates (Gemini 2.5 Flash assumed pricing: input ~\$0.30 / 1M tokens, output ~\$2.50 / 1M tokens; subject to Google's current rates; a free tier typically covers early usage):

Feature	Input tokens	Output tokens	Cost / request
Listing draft (with images)	~850-1,500	~500-800	~\$0.0020
Conversational search	~300-500	~150-250	~\$0.0006
Negotiation advice	~250-400	~200-350	~\$0.0008

Usage assumptions (per monthly active user, MAU): 0.5 AI listing drafts, 6 AI searches, 0.4 negotiation suggestions = **~6.9 AI requests/MAU/month**, ~4,800 tokens/MAU/month, **~\$0.005 AI cost/MAU/month**. These are deliberately conservative; real cost depends on the seller/buyer mix and how often the heuristic fallback absorbs search traffic.

Users (MAU)	AI requests / mo	Tokens / mo	Monthly AI cost
100	~690	~0.48M	~\$0.49 (likely \$0 on free tier)
500	~3,450	~2.4M	~\$2.46
1,000	~6,900	~4.8M	~\$4.92
5,000	~34,500	~23.9M	~\$24.60
10,000	~69,000	~47.8M	~\$49.20
25,000	~172,500	~119M	~\$123
50,000	~345,000	~239M	~\$246
100,000	~690,000	~478M	~\$492

Takeaway: AI is a negligible cost driver at every stage (under ~\$500/mo at 100k MAU) because the app uses one efficient Flash model, on-demand only, with no background AI jobs and deterministic fallbacks.

Part 4 - Infrastructure Cost

Estimates in USD/month. Ranges reflect tier base fees plus usage; the dominant variable cost is **image bandwidth** (Supabase Storage egress + Vercel). Assumes a single small region, Supabase Pro from ~1k users, Vercel Pro throughout.

Users (MAU)	Vercel	Supabase (DB+Auth)	Storage + Bandwidth	Email (Resend)	AI	Total / mo
100	\$0-20	\$0 (Free)	\$0 (Free)	\$0 (Free)	~\$0	~\$0-20
500	\$20	\$0-25	~\$0	\$0	~\$2	~\$25-50
1,000	\$20	\$25 (Pro)	~\$0-10	\$0-20	~\$5	~\$50-80
5,000	\$20-40	\$25-40	~\$20-40	\$20	~\$25	~\$110-165
10,000	\$40-70	\$40-75	~\$40-80	\$20	~\$49	~\$190-295
25,000	\$90-150	\$90-130	~\$90-160	\$20-40	~\$123	~\$410-600
50,000	\$180-300	\$150-250	~\$180-320	\$40-90	~\$246	~\$800-1,200
100,000	\$350-600	\$300-600 (->Team)	~\$350-650	\$90	~\$492	~\$1,600-2,400

Notes: Vercel Hobby is free but not licensed for commercial use, so Pro (\$20 base) is assumed. Supabase Free covers ~50k auth MAU but only 1GB storage / limited egress; image-heavy growth pushes to Pro (\$25) then compute/egress add-ons. Resend Free = 3,000 emails/mo, then ~\$20 for 50k. Payments are not processed (no Stripe fees).

Part 5 - Scaling

100 users

- Everything on free/cheap tiers; single Postgres instance idles. No changes needed. Cost ~\$0-20/mo.
- Auto-scaling: Vercel functions and Supabase handle this trivially. Bottlenecks: none. Optimization: just ship.

1,000 users

- Move Supabase to Pro for headroom + daily backups. Vercel Pro fine. Bandwidth still small.
- Auto-scales: app compute, auth. Upgrade: Supabase plan. Expensive first: nothing material. Bottleneck: none yet. Optimize: add caching headers on Storage, enable ISR on listing/category pages.

10,000 users

- Image bandwidth and Postgres connections become the things to watch. Introduce connection pooling (Supabase pooler/PgBouncer) for serverless function fan-out.
- Auto-scales: Vercel functions/CDN. Upgrade: Supabase compute add-on; possibly Resend paid. Expensive first: storage egress (images). Bottleneck: per-request DB connections from serverless; unindexed search at volume. Optimize: ISR/edge cache for read-heavy pages, CDN cache-control on images, ensure indexes on hot query paths, batch saved-search alerts.

100,000 users

- Approaching Supabase Team tier; consider read replicas for read-heavy feed/search, and a dedicated/compute-optimized Postgres. Search should graduate from ILIKE+FTS to trigram (pg_trgm) and/or pgvector semantic search (infra already present).
- Auto-scales: edge/CDN, stateless functions. Upgrade: Postgres compute + replicas, Storage egress plan, email volume. Expensive first: bandwidth, then DB compute. Bottleneck: write contention on counters, realtime connection limits, search latency. Optimize: materialized/denormalized feeds, cache layer (Upstash Redis), move counters to async aggregation, rate limiting.

1,000,000 users

- Dedicated Postgres (or partition/shard hot tables: listings, messages), read replicas, CDN-first architecture, a managed cache (Redis), and a real search service (or pgvector at scale with proper indexing). Realtime fan-out may need dedicated channels/limits. Background workers (queue) for alerts, embeddings, email, moderation.
 - Auto-scales: edge/functions/CDN. Upgrade: database tier (largest cost), cache, search, observability (Sentry/Datadog become necessary). Expensive first: database compute + bandwidth. Bottleneck: Postgres write throughput, hot-row counters, realtime connections. Optimize: sharding/partitioning, async pipelines, aggressive edge caching, AI caching + smaller models, image lifecycle policies.
-

Part 6 - Investor Technical Summary

- **Why this architecture is modern:** Server-first React (Next.js 15 RSC + Server Actions) removes most bespoke API code; authorization lives in the database via RLS; AI is integrated behind a clean provider interface. It is a 2025-era serverless stack with minimal moving parts.
- **Why it is scalable:** The app tier is stateless and auto-scales on Vercel's edge; Postgres scales vertically then via pooling/replicas; storage/CDN scale natively. The same codebase serves 100 or 100,000 users with configuration, not rewrites.
- **Why Next.js:** One framework for SSR, routing, image optimization, SEO, and secure server mutations; first-class Vercel deploys; huge talent pool; excellent Core Web Vitals for a consumer marketplace.
- **Why Supabase:** Managed Postgres + Auth + Storage + Realtime in one product. RLS gives database-enforced security; PostGIS/pgvector/FTS are built in. It compresses a backend team's worth of infrastructure into a single managed service - fast to build, cheap to run early.
- **Why TypeScript:** End-to-end type safety from UI to data access reduces runtime defects, speeds onboarding, and makes refactors safe as the team grows.
- **Why Vercel:** Zero-config global deploys, preview environments per commit, edge CDN and image optimization, and tight Next.js integration - minimal DevOps overhead for a small team.
- **Why the AI abstraction layer is valuable:** All model calls go through one AiProvider interface. The concrete provider is chosen by an env var, so model/provider changes are a config + one-class change, not a refactor. It also localizes prompt/cost control.
- **Switching AI providers in future:** OpenAI and Claude branches are already templated in the factory; implementing the interface for a new provider (or a cheaper Gemini tier) swaps the backend with no changes to product code or UI.
- **Why operating costs stay low early:** Free/low tiers cover the first thousands of users; AI is on-demand with one efficient model and deterministic fallbacks (under ~\$5/mo at 1k MAU all-in for AI). No payment processing fees. Costs grow sub-linearly with users until bandwidth/DB compute dominate.
- **Why it suits a UAE-first, GCC-expansion marketplace:** Bilingual-ready schema (title_en/title_ar, Arabic FTS), emirate-aware data model, AED-native pricing, locale en_AE SEO, and region-agnostic managed infra that can add regions/languages without re-architecting. Trust features (verification ladder, reviews, reports, moderation) fit a high-trust GCC market.

Part 7 - Cost Optimizations

Optimization	What / where	Est. savings
AI result caching	Cache parsed search results by normalized query (and reuse common drafts); skip identical Gemini calls.	40-70% of AI search cost
Prompt optimization	Trim system prompts, shrink schemas, cap output length; fewer tokens per call.	10-30% of AI cost
Smaller models for simple tasks	Use a Flash-Lite / cheaper tier for search parse + negotiation (keep full model for vision drafts).	50-70% on those features
Image optimization	Already downscales to 1024/q0.7; add responsive sizes, AVIF, and stricter compression for thumbnails.	30-60% of image bandwidth
Database indexing	Ensure indexes on hot filters (status, category, emirate, price, published_at) and search paths; avoid table scans.	Lower DB compute / faster queries
Edge caching (ISR)	Statically cache + revalidate listing detail and category/feed pages; cut function invocations and DB reads.	50-80% of read DB load
CDN optimization	Long-lived cache-control + immutable URLs on Storage images; maximize CDN hit rate.	20-40% of egress
Storage optimization	Compress on upload, dedupe, and add lifecycle cleanup for orphaned/expired images.	20-40% of storage

Optimization	What / where	Est. savings
Rate limiting	Add per-IP/user limits (e.g. Upstash) on search/AI/upload to prevent abuse-driven cost spikes.	Caps worst-case cost
Background jobs	Batch saved-search alerts, embeddings, and emails off the request path (queue/cron) to smooth compute.	Smooths peaks; 10-20% compute

Combined, caching + ISR + image/CDN tuning typically remove the majority of the variable cost at 10k+ MAU, keeping all-in spend well under the upper ranges in Part 4.

Prepared from a file-level audit of the Query & Buy repository. Implemented/partial/planned states reflect the code at report date. Financial figures are estimates for planning, not guarantees; validate against live Vercel/Supabase/Google billing as usage data accrues.